

Project Implementation Guide - Memory

Generated at: 2026-04-30 04:08:38

Project Overview and Purpose

Technical implementation guide for graph generation, enrichment, workflow tracing, and report outputs.

Architecture Diagram

Architecture image is generated per input project in hosted developer guide architecture page.

Tech Stack and Structure

Python, NetworkX, ReportLab, MkDocs, OpenAI integration; pipeline + functions modules compose the system.

Data Flow and I/O

Input folder -> parse/build graph -> enrich -> workflow/scenarios -> PDF/JSON/docs outputs.

Run, Testing, and Deployment

Run with: `python main.py --folder C:\Users\jarvi\OneDrive\Desktop\Book`

`syntesizer\memory_input_clean\Memory`

Test with scenario JSON + `mkdocs --strict build`.

CI/CD and pre-push hooks enforce regeneration.

Error Handling, Limitations, Future Improvements

LLM fallback is enabled; large repos may increase runtime; generic text suppression and workflow precision can be further improved.

Governance and Push Enforcement (Operational)

Local pre-push gate: `.git/hooks/pre-push`

CI post-push workflow: `.github/workflows/docs-on-push.yml`

Docs are regenerated on each run and existing files are overwritten.

Input Folder Analyzed

`C:\Users\jarvi\OneDrive\Desktop\Book syntesizer\memory_input_clean\Memory`

Workflow Steps Snapshot

1. `step_1` - orchestrates workflow execution steps
2. `step_2` - orchestrates workflow execution steps
3. `step_3` - orchestrates workflow execution steps
4. `step_4` - Handles search stage transition
5. `step_5` - Handles cosine stage transition
6. `step_6` - Handles analyze stage transition
7. `step_7` - Handles to dict stage transition
8. `step_8` - Handles walk stage transition
9. `step_9` - loads and retrieves required data

10. step_10 - builds and composes output artifacts
11. step_11 - loads and retrieves required data
12. step_12 - Handles walk stage transition
13. step_13 - Predict intent scores using BERT semantic classification
14. step_14 - Pre-load the model into memory (call during startup if desired)
15. step_15 - Handles chat stage transition
16. step_16 - Handles recent stage transition
17. step_17 - Handles stats stage transition
18. step_18 - Handles interactions stage transition
19. step_19 - JSON graph data: center node + neighbors + labeled edges
20. step_20 - Interactive visual graph viewer using vis-network
21. step_21 - Test the intent classifier and router logic without running the full pipeline
22. step_22 - Returns general stats or config of the router
23. step_23 - Retrieves required data in get_tree
24. step_24 - Handles mcp search stage transition

Internal Modules Snapshot

- `chat.py`
- `debug.py`
- `graph.py`
- `router\_api.py`
- `tree.py`
- `config.py`
- `embeddings.py`
- `llm.py`
- `gemini\_provider.py`
- `openai\_provider.py`
- `sleep\_consolidation\_job.py`
- `start\_background\_jobs.py`
- `main.py`
- `server.py`
- `emotional\_analyzer.py`
- `compaction\_engine.py`
- `memory\_lifecycle.py`
- `reflect\_engine.py`
- `sleep\_cycle.py`
- `contradiction\_detector.py`
- `memory\_extractor.py`
- `memory\_validator.py`
- `attention\_engine.py`
- `decision\_engine.py`
- `episode\_engine.py`

Top Connected Functions

- `route` (query_router.py) -> connectivity 37
- `run` (orchestrator.py) -> connectivity 28

- `analyze\_emotional\_signals` (emotional_analyzer.py) -> connectivity 27
- `compact` (compaction_engine.py) -> connectivity 22
- `retrieve` (hybrid_retriever.py) -> connectivity 18
- `run\_once` (sleep_consolidation_job.py) -> connectivity 17
- `extract` (memory_extractor.py) -> connectivity 17
- `analyze` (attention_engine.py) -> connectivity 16
- `store` (supermemory.py) -> connectivity 14
- `reflect` (reflect_engine.py) -> connectivity 14
- `build\_from\_memories` (tree_indexer.py) -> connectivity 14
- `\_score\_factual` (intent_classifier.py) -> connectivity 12

Implementation Summary

Metric	Value
Modules discovered	54
Classes discovered	62
Functions discovered	160
Entry points detected	20
Workflow steps traced	24
Graph nodes (all)	815
Graph edges (all)	1288
Call edges	806
Import edges	247
External nodes	501
Entry points listed	20
Workflow steps listed	24